TEAM ID	SWTID-2026-1705
PROJECT TITLE	AUTOMATING EMPLOYEE DATA

REQUIREMENT ANALYSIS

Requirement analysis is a comprehensive process that goes beyond simply listing what stakeholders want. It is about transforming vague ideas into clear, actionable requirements that guide the entire project lifecycle. At its core, requirement analysis ensures that the solution being developed is both feasible and aligned with business objectives. It involves multiple stages such as elicitation, where requirements are gathered through interviews, surveys, and workshops; analysis, where conflicts and redundancies are resolved; and prioritization, where critical needs are distinguished from optional features. Requirements are then documented in a structured format, often in a Software Requirement Specification (SRS), which serves as a reference point for designers, developers, and testers.

A well-executed requirement analysis also emphasizes validation, ensuring that requirements are realistic, testable, and agreed upon by all stakeholders. This reduces the risk of scope creep,
miscommunication,
and costly rework during later phases of development. For example, in

developing an e-commerce platform, requirement analysis would not only capture functional needs like product search, cart management, and payment processing, but also non-functional aspects such as security, scalability, and performance.

DATA FLOW DIAGRAMS AND USER STORIES:

Requirement analysis, Data Flow Diagrams, and User Stories together form the backbone of effective system design and development.

Requirement analysis is the process of gathering, analyzing, and documenting the needs of stakeholders to ensure that the system being developed aligns with business goals and user expectations.

It involves eliciting requirements through interviews and surveys, classifying them into functional and non-functional categories, prioritizing them based on importance, and validating them to avoid misunderstandings or scope creep.

Once requirements are clear, Data Flow Diagrams (DFDs) are used to provide a visual representation of how data moves through the system.

A DFD illustrates the interaction between external entities, processes, data stores, and data flows, helping stakeholders understand the logical structure of the system.

For example, in an online shopping application, a DFD might show how

customer order details flow into the order processing system, get stored in the database, and trigger notifications to the warehouse.

Alongside this, User Stories capture requirements from the perspective of the end user in a simple narrative format, typically structured as “As a [role], I want [feature], so that [benefit].” These stories emphasize user needs and motivations, making them easy to understand and prioritize in agile development. For instance, a user story might state, “As a customer, I want to track my order in real-time so that I know when it will arrive.” While requirement analysis ensures clarity and completeness, DFDs provide a system-level view of processes and data movement, and User Stories highlight user-centered features. Together, they create a comprehensive framework that balances technical accuracy with usability, ensuring that the final product is both functional and valuable to its users.

SOLUTION REQUIREMENTS:

Introduction

Solution requirements define the detailed capabilities, qualities, and constraints that a system must satisfy to meet business objectives and stakeholder expectations. They act as the bridge between high-level business requirements and the technical specifications used by developers and testers.

Categories of Solution Requirements

a) Functional Requirements

- Describe what the system should do.
- Focus on features, operations, and workflows.

- Examples:
  - The system shall allow users to register and log in using email and password.
  - The system shall generate monthly sales reports in PDF format.

b) Non-Functional Requirements

- Describe how the system should perform.
- Focus on quality attributes such as performance, security, usability, and scalability.
- Examples:
  - The system shall respond to user queries within 2 seconds.
  - The system shall encrypt all sensitive data using AES-256.

c) Transition Requirements

- Define temporary needs required to move from the current state to the new solution.
- Examples:
  - All customer data must be migrated from the legacy database before launch.
  - Staff must be trained on the new system within two weeks of deployment.

Characteristics of Good Solution Requirements

- **Clear and Unambiguous**: Easily understood by all stakeholders.
- **Measurable and Testable**: Can be validated through testing.
- **Feasible**: Realistic within project constraints (time, budget, technology).
- **Prioritized**: Ranked by importance to guide development focus.
- **Consistent**: No contradictions between requirements.

Importance of Solution Requirements

- Provide a foundation for design, development, and testing.
- Ensure alignment between business goals and technical implementation.
- Reduce ambiguity and prevent scope creep.
- Improve communication between business stakeholders and technical teams

. ●Minimize costly rework by clarifying needs early.

Example (E-Commerce Platform)

- **Functional**: The system shall allow customers to add products to a shopping cart and proceed to checkout.

- **Non-Functional**: The system shall support 10,000 concurrent users without performance degradation.

- **Transition**: Legacy customer accounts must be imported into the new platform before go-live.

Conclusion

Solution requirements are the backbone of successful system development. By clearly defining functional, non-functional, and transition requirements, organizations can ensure that the final product is both technically sound and user-centered, delivering value to stakeholders while meeting long-term business objectives.

TECHNOLOGY STACKS

Introduction

A technology stack is the combination of programming languages, frameworks, databases, and tools used to build and run a software application. It defines how the front-end, back-end, and supporting

services work together to deliver functionality and performance.

Front-End Technologies

The front-end is the client-side interface that users interact with. Common technologies include **HTML, CSS, JavaScript**, and frameworks such as **React, Angular, and Vue.js**. These tools focus on user experience, responsiveness, and design.

Back-End Technologies

The back-end is the server-side logic that powers the application. It manages data, processes requests, and ensures business rules are applied. Popular back-end frameworks include **Node.js, Django (Python), Spring Boot (Java), and .NET Core (C#)**.

Database Layer

Databases store and manage application data. They can be relational (SQL-based) or non-relational (NoSQL). Examples include **MySQL, PostgreSQL, Oracle, and MongoDB**. The choice depends on scalability, complexity, and type of data.

Supporting Tools & Services

Modern applications rely on DevOps and cloud services for deployment, monitoring, and scalability. Tools like **Docker, Kubernetes, Jenkins** and platforms such as **AWS, Azure, and Google Cloud** ensure smooth operations and continuous integration.

Popular Technology Stacks

- **MEAN**: MongoDB, Express, Angular, Node.js
- **MERN**: MongoDB, Express, React, Node.js
- **LAMP**: Linux, Apache, MySQL, PHP
- **Django Stack**: Python, Django, PostgreSQL
- **Spring Boot Stack**: Java, Spring Boot, MySQL/PostgreSQL

Modern Trends

Emerging stacks include **Jamstack** (JavaScript, APIs, Markup) for fast static sites, **Serverless architectures** using AWS Lambda or Azure Functions, and **cross-platform mobile stacks** like **Flutter** and **React Native** for building apps across iOS and Android.

Choosing the Right Stack

The choice of technology stack depends on:

- **Project Type** (web, mobile, enterprise, AI/ML).
- **Scalability Needs** (number of users, data volume).
- **Team Expertise** (developer skills).
- **Community Support** (documentation and resources).
- **Cost & Licensing** (open-source vs proprietary).

Conclusion

Technology stacks form the foundation of software development. By carefully selecting front-end, back-end, database, and supporting tools, organizations can build applications that are scalable, secure, and user-friendly, ensuring long-term success.